TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ



UÇAN ARABA YARIŞMASI

FİNAL TASARIM RAPORU

İLERİ KATEGORİ

TAKIM ADI: VENTUS

TAKIM ÜYELERİ:

Burak ERDEM

Muhammed Said SEVİNÇ

Afşin DEMİRÖZ

Ali Özgür POLAT

İçindekiler

Soru1 Hazırlanan senaryolarda kullanılan sensörler ve programlama dili ile ilgili aşağıdaki soruları cevaplayınız. 4

Soru1.1 Kod yazarken kullandığınız dil ve özelliklerini açıklayınız. 4

Soru1.2 Bu dil yerine başka hangi programlama dilini tercih ederdiniz? Nedeni ile açıklayınız. 4

Soru1.3 Barometre sensörünün nasıl çalıştığını açıklayınız. GNSS ile barometre irtifa verilerinin farkı nedir? 4

Soru1.4 İvmeölçer ve dönüölçer sensörlerinde genel hata karakteristikleri nelerdir? Bu karakteristiklerin özelliklerini açıklayınız. 4

Soru 2 Senaryo 1'de, hava aracının B noktasına gidişi için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız. 6

Soru2.1 Hava aracının gitmesi gereken yönü hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 6

Soru2.2 Hava aracının gitmesi gereken koordinatları hangi veri yapısından öğrendiniz? 8

Soru2.3 Hava aracının inişe geçmesi için gerekli koşul seçimleriniz nelerdir? 9

Soru2.4 İnişte hangi sensörü irtifa tespiti için kullandınız? 9

Soru2.5 İnişte yere değme hızınızı kaç m/s olarak ayarladınız? Nasıl? 9

Soru 3 Senaryo 4'te, yasak bölgeden kaçış için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız. 10

Soru3.1 Yasak bölgeleri tespit ettiğiniz veri yapısı nedir? 10

Soru3.2 Bu bölgelere girmemek için tasarladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız 10

Soru 4 Senaryo 7'de, trafikte olan diğer uçan arabaları gözeterek kaza yapmadan uçmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız. 12

Soru4.1 Diğer hava araçlarının konumlarını hangi veri yapısından öğrendiniz? 12

Soru4.2 Hava araçları ile kaza yapmamak için nasıl bir algoritma tasarladınız? Algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 12

Soru 5 Senaryo 8'de, en yakın hastaneyi bulmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız. 17

Soru5.1 Hastanelerin olduğu veri yapısı nedir? 17

Soru5.2 En yakın hastanenin seçimini yaptığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 17

Soru 6 Senaryo 10 içerisinde GNSS arızası yaşandığında inişi tamamlayan algoritma için aşağıdaki soruları cevaplayınız. 17

Soru6.1 GNSS arızasını hangi veri yapısından tespit ettiniz? 17

Soru6.2 Bu arıza sonrası hava aracının irtifasını hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 18

Soru 7 Senaryo 11 içerisinde GNSS arızası yaşandığında seyir yapmaya devam eden ve görevi tamamlamayı sağlayan algoritma için aşağıdaki soruları cevaplayınız. 19

Soru 7.1 Bu arıza sonrası hava aracının pozisyonunu(yatay) hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 19

Soru 8 Senaryo 12'de GNSS arıza durumunu tespit eden algoritma için aşağıdaki soruları cevaplayınız. 21

Soru 8.1 GNSS karıştırma (jamming) ve yanıltma (spoofing) nedir? Farkları nelerdir? 21

Soru 8.2 GNSS yanıltma (spoofing) durumunu tespit ettiğiniz algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 22

Soru 9 Senaryo 13'te inişi tamamlamak için kullanılacak sensör seçimini yaptığınız algoritma için aşağıdaki soruları cevaplayınız. 22

Soru 9.1 İniş için hangi sensörü irtifa kaynağı olarak seçtiniz? Neden? 22

Soru 9.2 Diğer sensörleri neden seçmediniz? 23

Soru 9.3 Bu seçimi yapan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 23

Soru 10 Senaryo 15'te manyetometre veri kaybı yaşandığında gidilmesi gereken hedefe hava aracının yönelmesini sağlayan algoritma için aşağıdaki soruları cevaplayınız. 24

Soru 10.1 Manyetometre veri kaybını nasıl tespit ettiniz? 24

Soru 10.2 Hava aracının gitmesi gereken yönü hesaplayan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 25

Soru 10.3 Hava aracının gittiği yönü hesaplayan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız. 27

Soru 11 Senaryo 16'da şarj durumunu gözeterek şarj istasyonlarını tespit ettiğiniz ve hava aracını oraya yönlendirdiğiniz algoritma için aşağıdaki soruları cevaplayınız. 28

Soru 11.1 Batarya durumlarını hangi veri yapısından öğrendiniz? 28

Soru 11.2 Batarya durumunun hangi gerilim eşiğinde şarj istasyonuna inilmesi gerektiğini seçtiniz? Neden? 28

Soru 11.3 Şarj istasyonlarının olduğu veri yapısı nedir? 28

Kaynakça 29

Soru1 Hazırlanan senaryolarda kullanılan sensörler ve programlama dili ile ilgili aşağıdaki soruları cevaplayınız.

Soru1.1 Kod yazarken kullandığınız dil ve özelliklerini açıklayınız.

Kodlarımızı yazarken Python dilini kullanmayı tercih ettik. Bunun sebeplerini dilin özelliklerini açıklayarak anlatalım. Python dili diğer dillerin aksine basit bir Syntax'a sahiptir. Bu özelliği onu kolay kullanılabilir ve öğrenilebilir bir dil haline getirmektedir. İşlerimizi hızlandırmak ya da kritik bir kod parçasını ana kodumuza eklemek istediğimizde örneğin C dilinde yazılan bir kod parçasını alıp Python dilindeki ana kodumuza ekleyebilir ve sorunsuz bir şekilde çalıştırabiliriz. Standart kütüphaneler olarak ele aldığımızda Python çok geniş bir kütüphane hacmine sahiptir. Bilimsel işlemlerden görüntü işleme ve makine öğrenmesine kadar birçok alanda bu kütüphaneleri kullanarak bize işlem kolaylığı sağlar.

Soru1.2 Bu dil yerine başka hangi programlama dilini tercih ederdiniz? Nedeni ile açıklayınız.

Alternatif bir dil seçeneği olarak C programlama dilini tercih edebiliriz. Bunun başlıca sebepleri ise dilin basit ve temel bir dil olmasıdır. Temel dilden anlatmak istediğimiz C programlama dilinin birçok açık kaynak kodlu projede kullanılıyor olması ve bunun bize öğrenim ve yorumlama açısında kaynak sunmasıdır. Bunun dışında C için en güvenilir programlama dili diyebiliriz. Kodun hazırlanmasında hemen hemen her şeyi yazara bırakan bu dil sayesinde bilgisayar ya da derleyici kaynaklı hatalardan kaçınmış oluruz.

Soru1.3 Barometre sensörünün nasıl çalıştığını açıklayınız. GNSS ile barometre irtifa verilerinin farkı nedir?

Barometre sensörü barometrik veya havanın atmosferik basıncını ölçmek için kullanılır. Bu işlemi belirli bir alandaki kuvvetten oluşan gerilimi bulmak için bir kuvvet toplayıcı kullanarak yapar. Mekanik gerilme uygulandığında yarı iletken ya da metalin üzerindeki elektrik gerilimi değişimi ölçülür. Voltaj çıkışı ya da veri analog şeklindedir. Daha sonra konsolda işlenerek dijitale dönüştürülür. Bu sensörden irtifa alırken temel mantığımız irtifa arttıkça atmosfer basıncındaki düşmenin oranını bulmaktır. Yeryüzü ile aradaki basıncı farkı hesaplanarak irtifayı elde ederiz. Gnss kullanarak irtifa belirlerken ise geriden kestirme yöntemini kullanırız. Bu yöntem uydu ile alıcı arasındaki uzaklık uydudan çıkan sinyalin alıcıya ulaşması için geçen sürenin ışık hızı ile çarpılması sonucu alıcının dünya üzerindeki konumunun bulunması üzerinedir.

Soru1.4 İvmeölçer ve dönüölçer sensörlerinde genel hata karakteristikleri nelerdir? Bu karakteristiklerin özelliklerini açıklayınız.

İvmeölçer ve dönüölçer sensörleri ataletsel sensörlerdir. Ataletsel sensörlerde iki tip hata kaynağı bulunmaktadır. Bunlardan ilki deterministik hatalar diğeri ise olasılıksal hatalardır. Deterministik hatalar olasılıksal hatalara nazaran bulunması ve çözülmesi daha kolaydır. Deterministik hatalar sabit kayma, orantı katsayısı kayma ve eksenel kaçıklık hatalarıdır. Olasılıksal hatalar ise sabit kayma kararsızlığı, orantı katsayısı kararsızlığı ve rastgele sensör gürültüsüdür. Bunlara ek olarak dönüölçerde deterministik hata sınıfına giren ivmeye bağlı sabit kayma hatası bulunmaktadır. Hataların özelliklerinden bahsedecek olursak:

- Sabit kayma hatası sensörlere herhangi bir giriş olmadığı halde alınan çıkıştır. Sabit terim, ısı ile oluşan değişimler gibi farklı bileşenleri olabilir. İvmeölçerde g ya da mg(mili g), dönüölçerde ise derece/saniye ya da derece/saat olarak ifade edilir.
- Orantı katsayısı kayma hatası sensörün çıkışındaki değer ile girişindeki değer arasında oluşan orantı katsayısı ideal orantı katsayısı olan 1'den farkıdır. Bu sayede sensörlerin hassasiyeti belirlenebilir. Her iki sensörde de ppm(part per million) olarak ifade edilir.
- Eksenel kaçıklık hatası ivmeölçerde dahili ve harici olarak ikiye ayrılır. Harici hatada sensörler ve eksenler birbirlerine tam dik şekilde yerleştirilmemiş olabilir. Bu durum her iki sensör içinde hata kaynağıdır. Dahili durumda ise ivmeölçerin üretiminden kaynaklı bir eksenel bozukluk olabilir. Her iki sensörde de mrad(mili radyan) olarak ifade edilir.
- Sabit kayma kararsızlığı hatası sensörlerin çalışma sırasındaki değişimlerinden kaynaklanan hatalardandır. Sensör çalıştığı sürece bu bileşen değişim gösterebilir. İvmeölçerde g/ √Hz, dönüölçerde ise derece/saniye√Hz ya da derece/saat√Hz olarak ifade edilir.
- Orantı katsayısı kararsızlığı hatası sensörlerin etkin olduğu süre boyunca orantı katsayısındaki değişimleri ifade eder. Bu hatanın anlaşılabilmesi için uzun süreli testler yapılmalıdır.
- Rastgele sensör gürültüleri yüksek frekanslı sensör gürültü terimleri (nicemleme gürültüsü, Gauss-Markov gürültüsü vb.) tarafından oluşturulur. İvmeölçerde g/√Hz, dönüölçerde ise derece/saniye√Hz ya da derece/saat√Hz olarak ifade edilir.
- İvmeye bağlı sabit kayma hatası dönüölçerin doğrusal ivmeye karşı hassasiyetinden oluşur. Bu etki sensörün çıkışında kaymaya sebep olur ve aralarında doğru orantı bulunmaktadır. Bu hata (derece/saniye)/g olarak ifade edilir.

Soru 2 Senaryo 1'de, hava aracının B noktasına gidişi için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız.

Soru2.1 Hava aracının gitmesi gereken yönü hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

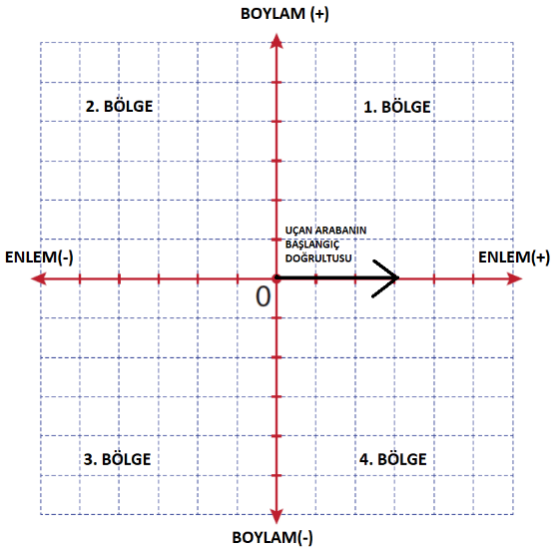
ALGORİTMA	AÇIKLAMA
yatay=abs(varis_bolgesi.enlem-cezeri.gnss.enlem) dikey=abs(varis_bolgesi.boylam-cezeri.gnss.boylam)	"varis_bolgesi" değişkeni hedefimizin bulunduğu nokta ile ilgili enlem, boylam, irtifa gibi değerleri taşımaktadır. Bulunduğumuz konumdan hedefe olan yatay ve dikey uzunlukları mutlak değer içinde yandaki gibi hesaplıyoruz. Mutlak değer fonksiyonuna alma sebebimiz bu değişkenlerin uzunluk değeri olmalarından dolayıdır.
hipotenus=math.sqrt((yatay**2)+(dikey**2)) cos_deger=(yatay/hipotenüs) radyan_aci=math.acos(cos_deger) derece_aci=(radyan_aci*180.0)/math.pi	Hesapladığımız "yatay" ve "dikey" değişkenlerini kullanarak hipotenüsü yandaki kodun ilk satırında hesaplıyoruz. Aracımız bir koordinat düzleminde bulunduğu için aracımızı bu koordinat düzleminin merkezinde kabul ederek koordinat düzleminin x ekseni olan enlem ekseni ile yaptığı dar açıyı yandaki kod bloğundan geometri formüllerini kullanarak hesaplıyoruz. "math.acos()" fonksiyonu radyan cinsinden değer döndürdüğü için dereceye döndürmek için "derece_aci" olarak tekrar hesaplıyoruz. Bulduğumuz bu "derece_aci" değeri hedefin yatay ile yaptığı açıdır ve değeri her zaman pozitif ve 90'dan küçüktür.

```
 if cezeri.gnss.enlem < varis_bolgesi.enlem and cezeri.gnss.boylam < varis_bolgesi.boylam:
     bolge_donus=(4.0*(derece_aci))/345.0

   elif cezeri.gnss.enlem > varis_bolgesi.enlem and cezeri.gnss.boylam < varis_bolgesi.boylam:
     bolge_donus=(4.0*(180.0-derece_aci))/345.0

   elif cezeri.gnss.enlem > varis_bolgesi.enlem and cezeri.gnss.boylam > varis_bolgesi.boylam:
     bolge_donus=(4.0*(180.0+derece_aci))/345.0

   elif cezeri.gnss.enlem < varis_bolgesi.enlem and cezeri.gnss.boylam > varis_bolgesi.boylam:
     bolge_donus=(4.0*(360.0-derece_aci))/345.0
   hedefe_yonel(bolge_donus)
```



Hedefin Cezeri'ye göre koordinat düzleminin hangi bölgesinde olduğuna göre aracı döndürmemiz gereken açı değişmektedir. Bu kontrolü aracın bulunduğu konumun enlem ve boylam değerlerinin büyüklüğünün karşılaştırılmasıyla sağlıyoruz. Her bir koşul yapısı sırasıyla sol alttaki resimdeki bölgeleri ifade etmektedir. "bölge_donus" değişkeni aracın hedefe yönelmesi için dönmesi gereken süreyi ifade etmektedir. Bu süreyi de riders.ai simülasyon ortamında cezeri 4 saniyede 360 derece dönebiliyorsa bizim bulduğumuz açı değeri için kaç saniye dönmesi gerekir diye düşünerek oran orantı ile hesaplıyoruz. Bu mantıkla kodumuzu hazırladıktan sonra simülasyon ortamında denediğimizde bu değerin 345 derece olmasını daha uygun bulduk. "derece_aci" değişkenimiz enlem ile yapılan dar açı değeri olduğu için her bir bölge için ayrı bir hesaplama ile dönmesi gereken süreyi hesaplıyoruz. Son olarak dönüşü gerçekleştirecek olan fonksiyonumuz olan "hedefe_yonel()"fonksiyonuna "bolge_donus" değerimizi gönderiyoruz. Ve hedefimize yöneliyoruz.

```
def hedefe_yonel(bolge_donus):
  bas_acisi=cezeri.manyetometre.veri*57.2957
  hedef_aci=(345.0*bolge_donus)/4
  donus_acisi=abs(hedef_aci-bas_acisi)
  if donus_acisi>360:
     donus_acisi=donus_acisi-360
  if donus_acisi>=180:
    donus_suresi=(4*(360-donus_acisi))/345.0
    cezeri.don(-3.14)
    cezeri.bekle(donus_suresi)
  elif donus_acisi<180:
    donus_suresi=(4*donus_acisi)/345.0
    cezeri.don(3.14)
    cezeri.bekle(donus_suresi)
  cezeri.dur()
```

Bu fonksiyon tanımlamamız sayesinde hedefe en kısa dönüş miktarı ile sorunsuz bir şekilde dönebilmekteyiz.

“cezeri.manyetometre.veri” verisi cezerinin radyan cinsinden dönüş değerini vermektedir. Bu veri radyan cinsinden olduğu için dönüşüm değeri olan “180/pi” ye çarparak (57.2957) cezerinin hedefe yönelmeden önceki dönüş durumunu “bas_acisi” değişkenine aktarıyoruz. Ardından tersine bir işlem gerçekleştirerek bolge_donus değişkenindeki süreyi açı değerine çeviriyoruz ve “hedef_aci” değişkenine aktarıyoruz. Bu açı değeri cezeri 0.0 bas_acisi’na sahipken hedefe yönelmek için gereken açı değeridir.

Bu bulduğumuz 2 açı arasındaki farkın mutlak değerini alıp “donus_acisi” değişkenine aktarıyoruz. Bu açı değeri 1 açının negatif 1 açının pozitif değere sahip olmasından dolayı 360 dereceden büyük olabilir. Bu sebeple ilk koşul yapısında bu durumdan kurtuluyoruz. Ardından ikinci koşulda dönmemiz gereken açı değeri 180 dereceden büyükse dönmemiz gereken yönün tersine diğer koşul olan 180 dereceden küçükse dönmemiz gereken yönde dönüşleri gerçekleştiriyoruz. Bu sayede en kısa şekilde dönüşleri otonom olarak sağlıyoruz.

Soru2.2 Hava aracının gitmesi gereken koordinatları hangi veri yapısından öğrendiniz?

Hava aracının gitmesi gereken koordinatları "cezeri.hedefler" veri yapısından öğrenmekteyiz. Bu veri yapısı aktif olan senaryonun hedef olarak belirlediği bölgeyi dizi şeklinde döndürür. Senaryo 1 için hedefimiz ile ilgili veriler "cezeri.hedefler[0]" da bulunmaktadır.

Soru2.3 Hava aracının inişe geçmesi için gerekli koşul seçimleriniz nelerdir?

Hava aracının inişe geçmesini sağlamak için hedef noktaya enlem ve boylam olarak uzaklıklarımızı while yapısının koşulları olarak seçiyoruz. Kullandığımız değerler uzaklık değerleri olduğu için enlem ve boylam farkları için mutlak değer fonksiyonunu kullanıyoruz. Hedefe yöneldikten sonra kodumuz bu while yapısına giriyor. while'ın içinde sürekli cezeri.ileri.git(cezeri.hizli) komutu ile hedefe yaklaşmasını sağlıyoruz. Bu yaklaşma enlem veya boylam farklarından biri 0.64 değerinden daha küçük olana kadar devam ediyor. Bu 0.64 değeri ise enlem veya boylam sabitken sabit olmayan tarafta düz olarak ilerlerken 1 saniyede değişen enlem veya boylam değeridir. while koşulundan çıktıktan sonra hedef noktaya 20 metre mesafe kalana kadar hızlı kalan mesafeyi yavaş bir şekilde inerek uçan arabanın inişi sağlanır. Hedefe ulaşıp hedefe geldiğinde durup inişe geçmesini sağlayan kodumuz şu şekildedir:

```
while ((abs(cezeri.gnss.enlem-varis_bolgesi.enlem)>=0.64) or
(abs(cezeri.gnss.boylam-varis_bolgesi.boylam)>=0.64)):
        cezeri.ileri_git(cezeri.hizli)
cezeri.dur()
while cezeri.gnss.irtifa>=varis_bolgesi.yukselti+20:
        cezeri.asagi_git(cezeri.hizli)
while cezeri.gnss.irtifa>=varis_bolgesi.yukselti:
        cezeri.asagi_git(cezeri.yavas)
```

Soru2.4 İnişte hangi sensörü irtifa tespiti için kullandınız?

İnişte GNSS sensörünü irtifa tespiti olarak kullandık. Bu sensörden gelen "cezeri.gnss.irtifa" verisi inişte irtifa tespitini sağlamak için kod yapısına dahil ettik.

Soru2.5 İnişte yere değme hızınızı kaç m/s olarak ayarladınız? Nasıl?

İnişte yere değme hızımızı 4 m/s olarak ayarladık. Yazdığımız kod ile indiğimiz bölgeye 20 metre mesafe kala hızımızı "cezeri.yavas" olarak ayarlayarak yavaş ve güvenli bir iniş sağlıyoruz. Bu işlemleri yaptığımız kod bloğu bu şekildedir:

```
while cezeri.gnss.irtifa>=varis_bolgesi.yukselti+20:
        cezeri.asagi_git(cezeri.hizli)
while cezeri.gnss.irtifa>=varis_bolgesi.yukselti:
        cezeri.asagi_git(cezeri.yavas)
```

Soru 3 Senaryo 4'te, yasak bölgeden kaçış için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız.

Soru3.1 Yasak bölgeleri tespit ettiğiniz veri yapısı nedir?

Yasaklı bölgelerin tespiti için cezeri.harita.ucusa_yasakli_bolgeler komutunu kullanarak harita üzerinde yasaklı bölgelerin komutunu burada bulunan dizi üzerinden erişim sağlamaktayız.

Soru3.2 Bu bölgelere girmemek için tasarladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama
cezeri.harita.ucusa_yasakli_bolgeler .enlem cezeri.harita.ucusa_yasakli_bolgeler .boylam	Bu komutla yasaklı bölgelerin enlem ve boylam bilgilerini tespit etmekteyiz.
en_kucuk_enlem=50 for yasakli_bolge in cezeri.harita.ucusa_yasakli_bolgeler: if yasakli_bolge.enlem<=en_kucuk_enlem : en_kucuk_enlem=yasakli_bolge.enlem en_kucuk_boylam=50 for yasakli_bolge in cezeri.harita.ucusa_yasakli_bolgeler: if yasakli_bolge.boylam<=en_kucuk_boylam : en_kucuk_boylam=yasakli_bolge.boylam en_buyuk_enlem=-50	Yan taraftaki kod paçası sayesinde yasaklı bölgenin köşe noktalarını tespit ediyoruz. Tespit ettiğimiz bu köşe noktalarını kullanarak aracın bu noktaları kullanarak yasaklı bölgeye girmesini engelliyoruz.

for yasakli_bolge in cezeri.harita.ucusa_yasakli_bolgeler: if yasakli_bolge.enlem >= en_buyuk_enlem : en_buyuk_enlem=yasakli_bolge.enlem en_buyuk_boylam=-50 for yasakli_bolge in cezeri.harita.ucusa_yasakli_bolgeler: if yasakli_bolge.boylam >= en_buyuk_boylam : en_buyuk_boylam=yasakli_bolge.boylam en_buyuk_boylam += 1 en_buyuk_enlem += 1 en_kucuk_boylam = en_kucuk_boylam-1 en_kucuk_enlem = en_kucuk_enlem-1	Bulunan enlem ve boylama +1 veya -1 eklenmesinin sebebi yasaklı bölgenin tespit edilen köşe noktalarından 1 er metre dışındaki noktaya giderek yasaklı bölgeden kaçınmaktır.
YeniTip=namedtuple('A_noktasi', "enlem boylam") s sag_ust_kose=YeniTip(en_buyuk_enlem,en_buyuk_boylam) sol_ust_kose=YeniTip(en_kucuk_enlem,en_buyuk_boylam) sag_alt_kose=YeniTip(en_buyuk_enlem,en_kucuk_boylam) sol_alt_kose=YeniTip(en_kucuk_enlem,en_kucuk_boylam)	Bu kısımda struct yapısını kullanıyoruz. Struct birbirleriyle ilişkili değişkenleri bir araya getirerek yeni bir tür olarak kullanma olanağı verir. Bizde bundan yararlanarak köşe noktalarının enlem ve boylam bilgilerini bir arada kullanabildiğimiz bu yapıyı oluşturduk Bunu kullanabilmek için yukarıdaki kütüphanelerimiz arasına “from collections import namedtuple” ekledik.
yatay=sag_ust_kose.enlem-cezeri.gnss.enlem dikey=sag_ust_kose.boylam-cezeri.gnss.boylam hipotenus1=math.sqrt((yatay**2)+(dikey**2)) yatay=sol_ust_kose.enlem-cezeri.gnss.enlem dikey=sol_ust_kose.boylam-cezeri.gnss.boylam hipotenus2=math.sqrt((yatay**2)+(dikey**2))	Tespit edilen noktalardan araca en yakın olanına hareket etmektedir ve yasaklı bölgeye girmeden etrafından dolanarak en kısa rotayı oluşturuyor bu sayede yasaklı

yatay=sag_alt_kose.enlem-cezeri.gnss.enlem
dikey=sag_alt_kose.boylam-cezeri.gnss.boylam
hipotenus3=math.sqrt((yatay**2)+(dikey**2))
 	
yatay=sol_alt_kose.enlem-cezeri.gnss.enlem
dikey=sol_alt_kose.boylam-cezeri.gnss.boylam
hipotenus4=math.sqrt((yatay**2)+(dikey**2))
 	
en_kucuk_hipotenus=100
if hipotenus1 <= en_kucuk_hipotenus:
  en_kucuk_hipotenus=hipotenus1
if hipotenus2 <= en_kucuk_hipotenus:
  en_kucuk_hipotenus=hipotenus2
if hipotenus3 <= en_kucuk_hipotenus:
  en_kucuk_hipotenus=hipotenus3
if hipotenus4 <= en_kucuk_hipotenus:
  en_kucuk_hipotenus=hipotenus4
	bölgeye girmeden en kısa rota ile hedefine ulaşmaktadır.
	
	Yan taraftaki kodda araca en yakın olan köşe noktası if yapısını kullanarak hipotenüslerinin kıyaslanması sonucu önceden olabilecek maksimum değeri verdiğimiz "en_kucuk_hipotenus" değerine hipotenüsler ile karşılaştırarak en kısa hipotenüs bulunur ve en yakın köşe noktası tespit edilir.

Soru 4 Senaryo 7'de, trafikte olan diğer uçan arabaları gözeterek kaza yapmadan uçmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız.

Soru4.1 Diğer hava araçlarının konumlarını hangi veri yapısından öğrendiniz?

Diğer hava araçlarının konumlarını "cezeri.trafik" dizisinden öğreniyoruz. "for trafik in cezeri.trafik:" döngü yapısıyla "cezeri,.trafik" dizisinin içindeki araçların konumlarını döngü içerisinde "trafik.enlem" ,"trafik.boylam" ile hava araçlarının konumlarını öğreniyoruz. For döngüsü içerisinde bu veri yapılarını inceleyerek araçların gittiği konumları inceleyebiliriz.

Soru4.2 Hava araçları ile kaza yapmamak için nasıl bir algoritma tasarladınız? Algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

ALGORİTMA	AÇIKLAMA

for a in range(100):
for trafik in cezeri.trafik:
if arac_kontrol==1:
if trafik.enlem<=arac1_enlem_kucuk :
arac1_enlem_kucuk=trafik.enlem
if trafik.boylam<=arac1_boylam_kucuk :
arac1_boylam_kucuk=trafik.boylam
if trafik.yukselti<=arac1_yukselti_farki_kucuk :
arac1_yukselti_farki_kucuk=trafik.yukselti
if trafik.enlem>=arac1_enlem_buyuk :
arac1_enlem_buyuk=trafik.enlem
if trafik.boylam>=arac1_boylam_buyuk :
arac1_boylam_buyuk=trafik.boylam
if trafik.yukselti>=arac1_yukselti_farki_buyuk :
arac1_yukselti_farki_buyuk=trafik.yukselti
if arac_kontrol==2:
if trafik.enlem<=arac2_enlem_kucuk :
arac2_enlem_kucuk=trafik.enlem
if trafik.boylam<=arac2_boylam_kucuk :
arac2_boylam_kucuk=trafik.boylam
if trafik.yukselti<=arac2_yukselti_farki_kucuk :
	Araç kalkmadan önce simülasyon ortamında bulunan diğer araçların hangi konumlarda gezdiğini bilmesi gerekir. Bu yüzden kalkmadan önce araçların geçtiği noktaların sınırlarını öğrenmesi için senaryodaki uçan arabaların nerelere gittiği kontrol ediyor ve sınır noktaları kalkmadan buluyor. Bu senaryo özelinde çözümümüz bu şekilde, gerçek hayatta bu veriler trafik yaratan uçan araçların rotaları belli olduğu için otomatik olarak aktarılacaktır. Üç adet uçan araba olduğu için bu üç uçan arabanın maksimum ve minimum enlem ve boylam bilgileri yukarıdaki for döngüsünün içinde belirleniyor. Belirlediğimiz bu 3 araç verisinin ortak sınırlarını belirlemek için tekrar bir işlemden geçiriyoruz. Bu

```
            arac2_yukselti_farki_kucuk=trafik.yukselti
        if trafik.enlem>=arac2_enlem_buyuk :
            arac2_enlem_buyuk=trafik.enlem
        if trafik.boylam>=arac2_boylam_buyuk :
            arac2_boylam_buyuk=trafik.boylam
        if trafik.yukselti>=arac2_yukselti_farki_buyuk :
            arac2_yukselti_farki_buyuk=trafik.yukselti
    if arac_kontrol==3:
        if trafik.enlem<=arac3_enlem_kucuk :
            arac3_enlem_kucuk=trafik.enlem
        if trafik.boylam<=arac3_boylam_kucuk :
            arac3_boylam_kucuk=trafik.boylam
        if trafik.yukselti<=arac3_yukselti_farki_kucuk :
            arac3_yukselti_farki_kucuk=trafik.yukselti
        if trafik.enlem>=arac3_enlem_buyuk :
            arac3_enlem_buyuk=trafik.enlem
        if trafik.boylam>=arac3_boylam_buyuk :
            arac3_boylam_buyuk=trafik.boylam
        if trafik.yukselti>=arac3_yukselti_farki_buyuk :
            arac3_yukselti_farki_buyuk=trafik.yukselti
        arac_kontrol=0
    arac_kontrol=arac_kontrol+1
  cezeri.bekle(0.30)
yatay=varis_bolgesi.enlem-cezeri.gnss.enlem
```

işlemdeki amaç trafik olan bölgelerin belirlenip bir kare oluşturarak uçan aracın bu karenin dışından giderek hedefe ulaşmasını hedefliyoruz. Gitmemiz gereken nokta bu karenin içinde kalabilir. Bu sebeple hedefe giderken ekstra bir koruma yöntemimiz daha bulunmaktadır. Aşağıda o yöntemden bahsediyor olacağız. En küçük nokta ve en büyük noktaları yandaki if yapıları ile oluşturuyoruz.

Bu işlemler sonucunda köşe noktalarının enlem ve boylam verilerine ulaşıyoruz. Ulaştığımız bu köşe noktaları verilerini "hedefe_git()" fonksiyonuna uyarlamak için python için kullanılan metottan faydalanıyoruz. Bu metodu kullanma amacımız hedefe gitmek için yazdığımız fonksiyonun tek değişken almasıdır.

dikey=varis_bolgesi.boylam-cezeri.gnss.boylam hipotenus=math.sqrt((yatay**2)+(dikey**2)) YeniTip=namedtuple('A_noktasi', "enlem boylam ") en_k_e= [arac1_enlem_kucuk,arac2_enlem_kucuk,arac3_enlem_kucuk] en_k_b= [arac1_boylam_kucuk,arac2_boylam_kucuk,arac3_boylam_kucuk] en_b_e= [arac1_enlem_buyuk,arac2_enlem_buyuk,arac3_enlem_buyuk] en_b_b= [arac1_boylam_buyuk,arac2_boylam_buyuk,arac3_boylam_buyuk] en_k_e.sort() en_k_b.sort() en_b_e.sort(reverse=True) en_b_b.sort(reverse=True) KK=YeniTip(en_k_e[0]-1,en_k_b[0]-1) KB=YeniTip(en_k_e[0]-1,en_b_b[0]+1) BK=YeniTip(en_b_e[0]+1,en_k_b[0]-1) BB=YeniTip(en_b_e[0]+1,en_b_b[0]+1)	Bu sebeple bulduğumuz enlem ve boylam verilerini structure yapısına benzer bir yapıyla birleştirerek kodda kolaylık sağlayacak tek bir değişken oluşturuyoruz. Oluşturduğumuz bu değişkenleri kullanarak trafik alanının çevresinden geçebiliyoruz. Şimdi aşağıda araç hareket ederken tehlikeli uçuş yaşamamak için yazdığımız koddan bahsedeceğim.
if varis_bolgesi.enlem == cezeri.gnss.enlem : kontrol_e= 0 if varis_bolgesi.boylam == cezeri.gnss.boylam : kontrol_b= 0 if varis_bolgesi.enlem>cezeri.gnss.enlem : kontrol_e = 0.65 if varis_bolgesi.enlem<cezeri.gnss.enlem : kontrol_e= -0.65 if varis_bolgesi.boylam>cezeri.gnss.boylam : kontrol_b = 0.65	Öncelikle if yapılarından sonraki while ın altındaki kodlar while koşuluna bağlıdır. Araç hedefe giderken bu senaryo özelinde önünde herhangi bir araç var mı diye sürekli kontrol halindedir. Eğer önünde bir araç varsa bu kod onu algılayarak tehlikeli bir uçuş yaşanmasına engel

```
  if varis_bolgesi.boylam<cezeri.gnss.boylam :
      kontrol_b= -0.65
while ((abs(cezeri.gnss.enlem-varis_bolgesi.enlem)>=0.65) or (abs(cezeri.gnss.boylam-varis_bolgesi.boylam)>=0.65)):
cezeri.ileri_git(cezeri.hizli)
    bolge=cezeri.harita.bolge(cezeri.gnss.enlem+
kontrol_e,cezeri.gnss.boylam+kontrol_b)
    bolge_sol_1=cezeri.harita.bolge(cezeri.gnss.enlem+
(kontrol_e*2),cezeri.gnss.boylam+kontrol_b*2)
    bolge_sol_2=cezeri.harita.bolge(cezeri.gnss.enlem+
((kontrol_e*2)+0.32),cezeri.gnss.boylam+(kontrol_b*2)+0.32)
    bolge_sol_3=cezeri.harita.bolge(cezeri.gnss.enlem+
(kontrol_e*3),cezeri.gnss.boylam+(kontrol_b*3))
    bolge_sag_1=cezeri.harita.bolge(cezeri.gnss.enlem+
(kontrol_e*2),cezeri.gnss.boylam+kontrol_b*2)
    bolge_sag_2=cezeri.harita.bolge(cezeri.gnss.enlem+
((kontrol_e*2)+0.32),cezeri.gnss.boylam+((kontrol_b*2)+0.32)
    bolge_sag_3=cezeri.harita.bolge(cezeri.gnss.enlem+
(kontrol_e*3),cezeri.gnss.boylam+(kontrol_b*3))
  while  bolge.trafik or bolge_sol_1.trafik  or bolge_sag_1.trafik or bolge_sol_2.trafik  or bolge_sag_2.trafik or bolge_sol_3.trafik or bolge_sag_3.trafik:
        cezeri.dur()
        cezeri.geri_git(cezeri.orta)
        cezeri.bekle(1)
```

olmaktadır. Çünkü bir araç ile karşılaştığı anda geriye doğru giderek aracın ona doğru yaklaşıp tehlikeye neden olmasını önlemektedir. Bu kontrolü GNSS sensöründen aldığımız verileri belli değerler ile arttırarak araç ilerlerken sürekli önünde uçan araç var mı diye kontrol etmesini sağlayarak yapıyoruz. En üstteki if yapısında aracın enlem ve boylam değişmelerini belirleyerek bolge değişkenlerinin aracın önünü solunu ve sağını kontrol etmesini sağlatıyoruz. Ve önünde araç olduğu anda aracımız durarak yandaki kod bloğunu işletiyor 1 saniye boyunca geri gelip döngü boyunca tekrar bölgeleri kontrol ettiriyoruz. Önünde araç olmadığında kodumuz hedefine gitmeye devam ediyor ve hedefe güvenli bir şekilde ulaşıyoruz.

Soru 5 Senaryo 8'de, en yakın hastaneyi bulmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız.

Soru5.1 Hastanelerin olduğu veri yapısı nedir?

"cezeri.harita.hastaneler" ifadesi harita üzerinde bulunan bütün hastanelerin saklandığı dizidir. Dizinin içinde bulunan hastane verilerine for döngüsü kullanarak " for hastane_bolge in cezeri.harita.hastaneler : " yapısını kullanarak her bir şarj istasyonun konum verilerine for döngüsünün içinde "hastane_bolge.enlem", "hastane_bolge.boylam", komutları ile ulaşmaktayız. Hastanelerin komut bilgilerine bu şekilde ulaşıyoruz.

Soru5.2 En yakın hastanenin seçimini yaptığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

Algoritma	Açıklama

en_kucuk_hipotenus=100
for hastane_bolge in cezeri.harita.hastaneler:
yatay=hastane_bolge.enlem-cezeri.gnss.enlem
dikey=hastane_bolge.boylam-cezeri.gnss.boylam
hipotenus=math.sqrt((yatay**2)+(dikey**2))
if hipotenus <= en_kucuk_hipotenus:
en_kucuk_hipotenus=hipotenus
secilen_hastane=hastane_bolge
	Yan tarafta görmüş olduğumuz algoritmada cezeri.harita.hastaneler dizisinin içindeki hastanelerin sırası ile konumunu alıp aracın kendi konumu ile kıyaslayıp ona en yakın olan hastanenin bilgilerini "secilen_hastane" değişkenin içine atamaktadır.
	Bu sayede araca en yakın olan hastanenin bütün bilgilerine ulaşabilmekteyiz.

Soru 6 Senaryo 10 içerisinde GNSS arızası yaşandığında inişi tamamlayan algoritma için aşağıdaki soruları cevaplayınız.

Soru6.1 GNSS arızasını hangi veri yapısından tespit ettiniz?

GNSS arıza durumunu tutan veri yapısı "cezeri.gnss.hata" verisidir. Bu veri yapısı GNSS arıza yaptığında "TRUE" diğer durumlarda "FALSE" değerini döndürmektedir.

Soru6.2 Bu arıza sonrası hava aracının irtifasını hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

ALGORİTMA	AÇIKLAMA

if cezeri.gnss.hata==1:
inis_yap_barometre(cezeri.hedefler[0])
if cezeri.gnss.hata==0:
inis_yap_gnss(cezeri.hedefler[0])
	Eğer GNSS de arıza var ise barometre sensörü ile eğer GNSS de arıza yok ise GNSS sensörü kullanılarak irtifa belirlenir ve hedefe iniş sağlanır. Bu senaryo için hedefe inmemizi sağlayan fonksiyon "iniş_yap_barometre" fonksiyonudur.

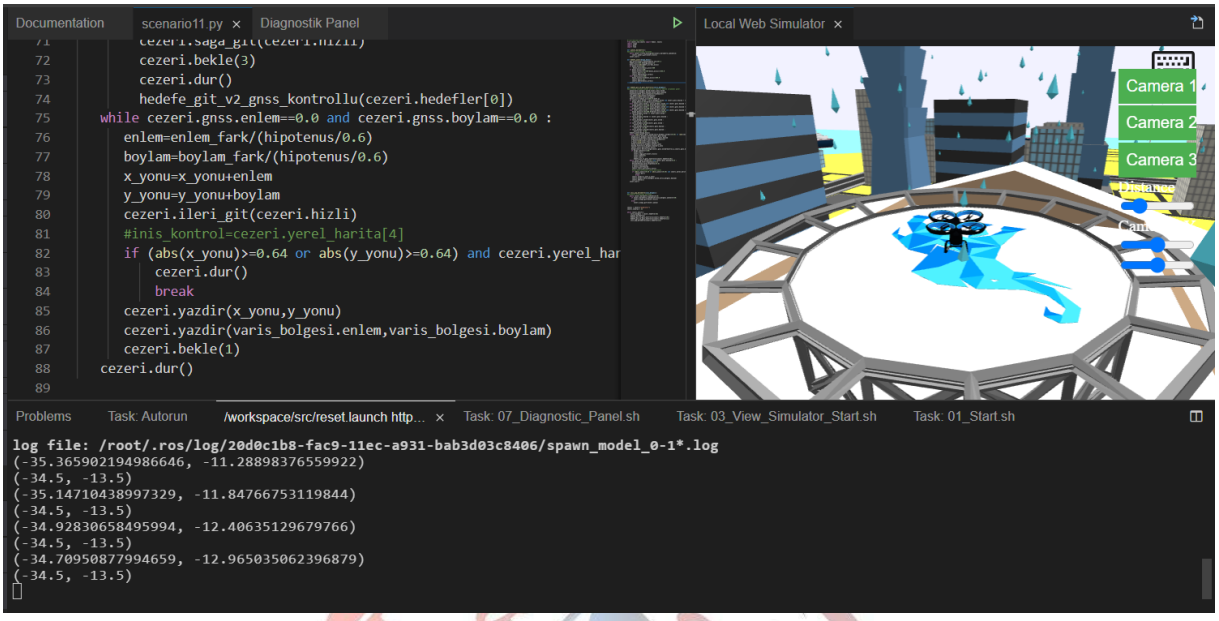
def inis_yap_gnss(inis_bolgesi):
#Gnss arizasinda kullanma
while cezeri.gnss.irtifa>=inis_bolgesi.yukselti+20:
cezeri.asagi_git(cezeri.hizli)
while cezeri.gnss.irtifa>=inis_bolgesi.yukselti:
cezeri.asagi_git(cezeri.yavas)
def inis_yap_barometre(inis_bolgesi):
#barometre arizasinda kullanma
while cezeri.barometre.yukseklik>=inis_bolgesi.yukselti+20:
cezeri.asagi_git(cezeri.hizli)
while cezeri.barometre.yukseklik>=inis_bolgesi.yukselti:
cezeri.asagi_git(cezeri.yavas)
	Bu kod blokları yukarıdaki kodun fonksiyonlarının tanımlamasıdır.10. senaryo da inişi gerçekleştiren fonksiyon "iniş_yap_barometre" fonksiyonudur. Barometre sensöründen irtifa verisi "cezeri.barometre.yukseklik" verisinden alınır. İndiğimiz bölgenin yükselti durumuna göre 20 metreye kadar hızlı bir şekilde son 20 metreyi yavaş bir şekilde inerek hedefe iniş sağlanır. Bu sayede GNSS arızası olsa bile hedefe güvenli bir şekilde iniş sağlanmış olur.

Soru 7 Senaryo 11 içerisinde GNSS arızası yaşandığında seyir yapmaya devam eden ve görevi tamamlamayı sağlayan algoritma için aşağıdaki soruları cevaplayınız.

Soru 7.1 Bu arıza sonrası hava aracının pozisyonunu(yatay) hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

ALGORİTMA	AÇIKLAMA
while ((abs(cezeri.gnss.enlem-varis_bolgesi.enlem)>=0.64) or (abs(cezeri.gnss.boylam-varis_bolgesi.boylam)>=0.64))and not cezeri.gnss.enlem==0.0 and not cezeri.gnss.boylam==0.0: yatay=varis_bolgesi.enlem-cezeri.gnss.enlem dikey=varis_bolgesi.boylam-cezeri.gnss.boylam hipotenus=math.sqrt((yatay**2)+(dikey**2)) x_yonu=round(cezeri.gnss.enlem,4) y_yonu=round(cezeri.gnss.boylam,4) enlem_fark=varis_bolgesi.enlem-x_yonu boylam_fark=varis_bolgesi.boylam-y_yonu	Aracımız bu senaryo özelinde hedefine doğru giderken GNSS sensöründen gelen enlem ve boylam bilgilerini sürekli kontrol etmektedir. Uçan arabamız GNSS verilerini alamadığı zaman otomatik olarak 0.0 değerlerini göndermektedir. O yüzden hata olduğunu anlamak için bu değerleri kullanıyoruz. Hedefe giderken aracımız sürekli hedef ile olan mesafesini hipotenüs değişkenine aktarmaktadır. Yandaki kod bloğu sayesinde araçta GNSS arizasi olmadan hemen önceki GNSS enlem, GNSS boylam ve hipotenüs değerlerini alabiliyoruz. Bu değerleri arıza olduğu durumda kullanmak için alıyoruz. Arıza olduğu durumda aşağıdaki kod bloğunu işletiyoruz.

while cezeri.gnss.enlem==0.0 and cezeri.gnss.boylam==0.0 : enlem=enlem_fark/(hipotenus/0.6) boylam=boylam_fark/(hipotenus/0.6) x_yonu=x_yonu+enlem y_yonu=y_yonu+boylam cezeri.ileri_git(cezeri.hizli) if (abs(x_yonu)>=0.64 or abs(y_yonu)>=0.64) and cezeri.yerel_harita[4].inis_bolgesi : cezeri.dur() break cezeri.yazdir(x_yonu,y_yonu) cezeri.yazdir(varis_bolgesi.enlem,varis_bolgesi.boylam) cezeri.bekle(1)	Eğer aracımız GNSS veri kaybından dolayı üstteki while dan çıkarsa bu while yapısının içine girmektedir. Yukarıdaki while dan aldığımız verileri bu while içinde sürekli olarak işleme sokarak aracımızın pozisyonunu 1 saniye aralıklarla ekrana yansıtabilmekteyiz. Ayrıca bu veriyi iniş noktamızı bulmak için bir koşul olarakta kullanmaktayız. Araç "cezeri.hizli " hızında ilerlerken 1 saniyede hedefe 0.6 metre yaklaşabilmektedir. Aradaki enlem ve boylam farkını kapatmak için geçmesi gereken süreyi "hipotenüs/0.6" işlemi ile bulmaktayız. Bu değeri ayrı ayrı "enlem_fark" ve "boylam_fark" değerimize böldüğümüzde 1 saniyede x ve y değerlerimizin artış veya azalış miktarlarını bulmaktayız. Bu değeri sürekli ekleyerek araç hareket ederken enlem ve boylam değerlerine düzgün ve doğru bir şekilde ulaşabilmekteyiz.



Yukarıdaki fotoğraf GNSS arizası sonrası hesaplayarak aldığımız enlem ve boylam bilgilerini ve gitmek istediğimiz hedefin enlem ve boylam bilgilerini ekrana yansıttığı bir örnektir. Ayrıca bu bilgileri iniş yapmak içinde kullanmaktayız.

Soru 8 Senaryo 12'de GNSS arıza durumunu tespit eden algoritma için aşağıdaki soruları cevaplayınız.

Soru 8.1 GNSS karıştırma (jamming) ve yanıltma (spoofing) nedir? Farkları nelerdir?

GNSS'e karşı yapılan en yaygın iki saldırı çeşidi jamming ve spoofing saldırılarıdır. Benzer bir mantığa sahip oldukları için karıştırılan bu iki saldırı çeşidini ayrı ayrı ele almak gerekirse, Jamming saldırısında benzer frekansta radyo dalgaları yayınlayarak kullanıcıya ulaşmaya çalışan radyo dalgaları bastırılır ve kabaca tabir etmek gerekirse bir gürültü oluşturur. Burada amaç kullanıcının iletişime ve verilere erişmesine engel olmaktır. Ancak Spoofing saldırılarında saldırgan kullanıcının sistemine erişim sağlayarak kontrol sahibi olmak ister. Spoofingde de benzer frekansda radyo dalgaları yayınlanır ve bu sayede saldırgan sisteme erişim sağlayabilir. Aradaki farkı özetlemek gerekirse yöntem ve araç olarak benzer görünen bu iki saldırı çeşidi amaç olarak birbirlerinden farklıdır. Jamming de sinyalin alınması engellenmek istenirken Spoofing de asıl amaç kontrolü ele geçirmektir.

Soru 8.2 GNSS yanıltma (spoofing) durumunu tespit ettiğiniz algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

ALGORİTMA	AÇIKLAMA
sayac=0 en_kucuk_irtifa=130 en_buyuk_irtifa=110 while sayac<100 : if cezeri.gnss.irtifa<=en_kucuk_irtifa : en_kucuk_irtifa=cezeri.gnss.irtifa if cezeri.gnss.irtifa>=en_buyuk_irtifa : en_buyuk_irtifa=cezeri.gnss.irtifa sayac=sayac+1 while en_kucuk_irtifa<=120.0 or en_buyuk_irtifa>=130 : cezeri.ileri_git(cezeri.yavas) inis_kontrol=cezeri.yerel_harita[4] if inis_kontrol.inis_bolgesi : cezeri.dur() cezeri.bekle(1) break	Hedefe giderken kullandığımız bu kontrol yöntemini GNSS de spoofing durumunu anlamak için kullanıyoruz. Spoofing anında uçan arabamızdaki tüm GNSS verilerimiz enlem, boylam ve irtifa değerleri yanlış değerler vermektedir. Bu kodda biz Spoofing durumunu irtifa değeri üzerinden anlıyoruz. İlk while ile 100'er kez irtifa değeri kontrol ettirip irtifa değerinin en küçük ve en büyük değerlerinde kayda değer bir değişme var mı diye kontrol ediyoruz. İrtifa değeri sürüş boyunca inişler hariç sabit olduğundan dolayı büyük miktardaki değişimlerde spoofing durumu olduğunu anlamamıza yarıyor. Spoofing durumunu anladıktan sonra hızımızı yavaşlatıp iniş noktasının kontrolünü yerel harita üzerinden kontrol ediyoruz. İniş noktasına geldiğimizde inişi güvenli bir şekilde gerçekleştiriyoruz.

Soru 9 Senaryo 13'te inişi tamamlamak için kullanılacak sensör seçimini yaptığınız algoritma için aşağıdaki soruları cevaplayınız.

Soru 9.1 İniş için hangi sensörü irtifa kaynağı olarak seçtiniz? Neden?

İniş irtifamızı belirlemede iki farklı yönteme başvurduk. Birinci yöntem olarak iniş sahasında yağmur olmadığı durumda lidar sensöründen irtifa verimizi alarak inişimizi sağladık. İniş sahasında

yağmur olması durumunda ise lidar sensörü çalışmayacağından dolayı z ekseninde alınan iniş hızından yol=hız*zaman formülünü kullanarak özel bir irtifa bulma fonksiyonu oluşturarak uçan aracımızın güvenli bir şekilde inişi tamamlamasını sağladık.

Soru 9.2 Diğer sensörleri neden seçmediniz?

Cezeri de irtifa hesaplamak için kullanılan sensörlerden olan GNSS, Barometre ve Radar sensörlerinde veri kaybı yaşandığından dolayı o sensörleri Senaryo 13 için kullanmamayı tercih ettik.

Soru 9.3 Bu seçimi yapan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

ALGORİTMA	AÇIKLAMA

if cezeri.gnss.hata==0:
inis_yap_gnss(varis_bolgesi)
elif cezeri.radar.hata==0:
inis_yap_radar(varis_bolgesi)
elif cezeri.barometre.hata==0:
inis_yap_barometre(varis_bolgesi)
 	
elif not varis_bolgesi.yagmur :
inis_yap_lidar(varis_bolgesi)
else :
mesafe = 124.0-varis_bolgesi.yukselti
zaman=(mesafe-20)/13.2
cezeri.asagi_git(cezeri.hizli)
cezeri.bekle(zaman)
cezeri.asagi_git(cezeri.yavas)
cezeri.bekle(10)
	Sırası ile tüm sensörlerin hata durumlarını kontrol ederek hata olmayan sensörü kullanarak iniş yapmasını sağlar. Eğer sensörlerin hepsinde bir arıza yaşanıyorsa else bloğu işletilir. Else bloğunda "cezeri.hizli" Vz ekseninde maksimum hız olarak 13.2 verdiğinden dolayı tahmini yükseltimizden iniş yapıcağımız bölgenin yükseltisini çıkararak iniş yapmamız gereken mesafeyi hesaplarız. Ardından bu mesafeden 20 metre çıkararak maksimum hızda kaç saniye boyunca aşağıya gideceğimizi hesaplarız. Son 20 metreyi de yavaş olarak inerek senaryoyu sorunsuz ve güvenli bir şekilde tamamlıyoruz.
	"iniş_yap_..." fonksiyonlarımız her sensör için aynıdır. Sadece aldığımız veri paketi değişmektedir. GNSS ve Barometre iniş fonksiyonları soru 6.2 de açıklanmıştır. Radar ve Lidar içinde bu fonksiyonlar aynı yöntem kullanılarak yazılmıştır.

Soru 10 Senaryo 15'te manyetometre veri kaybı yaşandığında gidilmesi gereken hedefe hava aracının yönelmesini sağlayan algoritma için aşağıdaki soruları cevaplayınız.

Soru 10.1 Manyetometre veri kaybını nasıl tespit ettiniz?

Manyetometredeki veri kaybını uçan aracımız Cezeri hedefine doğru ilerlerken sürekli yaptığımız kontroller sayesinde tespit ediyoruz. Port ekranına manyetometre arıza olduğunda "Manyetometre Arizali" yazısını yazdırıyoruz. Biz bu senaryo özelinde manyetometre arızalı olduğu durumda manyetometre hiç kullanmadan hedeflerine başarılı bir şekilde gidebilen bir hedefe_git() fonksiyonu oluşturduğumuz için senaryo 15 özelinde bu fonksiyonu kullanarak hedefe ulaşabiliyoruz. Manyetometre de bulunan hatayı tespit edip ekrana yazdırdığımız kod bloğu aşağıdaki gibidir. Aşağıdaki kodda kontrol ettiğimiz durum diğer soruda da bahsedeceğimiz durum olan aracın gittiği yön ile gitmesi gereken yön arasındaki farkın 5 dereceden fazla olması durumuna dayanmaktadır. Eğer 5 dereceden fazla bir fark olursa manyetometre de arıza olduğunu anlıyoruz ve ekrana yazdırıyoruz.

```
  while ((abs(cezeri.gnss.enlem-varis_bolgesi.enlem)>=0.64) or
(abs(cezeri.gnss.boylam-varis_bolgesi.boylam)>=0.64)):
    cezeri.ileri_git(cezeri.hizli)
    bas_acisi=(345.0*bolge_donus)/4
    if bolge_aci<0:
      bas_acisi=(-1*bas_acisi)
    cezeri.bekle(0.5)
    if abs(abs(cezeri.manyetometre.veri*57.3)-abs(bas_acisi))>=5:
       print("Manyetometre Arizali")
```

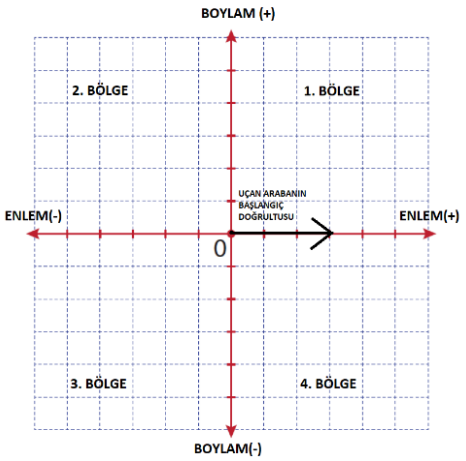
Yukarıdaki kod bloğu ile Senaryo 15 için yazdırdığımız "Manyetometre Arizali" ve bas_acisi değerlerini ekrana yazdırdığımız simülasyon ekranı resmimiz aşağıda bulunmaktadır. Bu resimde bas_acisi değerimiz yani uçan aracın dönüş açısı, manyetometre cihazından gelen veri (arızalı olduğu için 0.0 değerini döndürmektedir.) ve "Manyetometre Arizali" ifadesi bulunmaktadır.



Soru 10.2 Hava aracının gitmesi gereken yönü hesaplayan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

ALGORİTMA	AÇIKLAMA
def yon_duzelt(bolge_donus,bolge_aci): cezeri.don(-1*bolge_aci) cezeri.bekle(bolge_donus) cezeri.dur() cezeri.bekle(1)	Yandaki fonksiyonun amacı gitmek istenilen hedef her değiştiğinde kendini o hedefe döndüğü kadar geri döndürerek başlangıç konumu olan 0.0 dereceye her seferinde dönebilmesini sağlar.. Hedefe yönelirken döndüğü yönü ve dönüş süresini tutar. Hedef değiştiğinde kendini sıfırlar ve hedefe her seferinde doğru bir şekilde yönelebilmemizi sağlar.

yatay=varis_bolgesi.enlem-cezeri.gnss.enlem
dikey=varis_bolgesi.boylam-cezeri.gnss.boylam
hipotenus=math.sqrt((yatay**2)
+(dikey**2))
cos_deger=(abs(yatay)/hipotenus)
radyan_aci=math.acos(cos_deger)
derece_aci=(radyan_aci*180.0)/math.pi
 	
if cezeri.gnss.enlem < varis_bolgesi.enlem and cezeri.gnss.boylam < varis_bolgesi.boylam:
 	
bolge_donus=(4.0*(derece_aci))/345.0
 	
bolge_aci=3.14
 	
cezeri.don(bolge_aci)
 	
cezeri.bekle(bolge_donus)
 	
elif cezeri.gnss.enlem > varis_bolgesi.enlem and cezeri.gnss.boylam < varis_bolgesi.boylam:
 	
bolge_donus=(4.0*(180.0-derece_aci))/345.0
 	
bolge_aci=3.14
 	
cezeri.don(bolge_aci)
 	
cezeri.bekle(bolge_donus)
 	
elif cezeri.gnss.enlem > varis_bolgesi.enlem and cezeri.gnss.boylam > varis_bolgesi.boylam:
 	
bolge_donus=(4.0*(360-(180.0+derece_aci)))/345.0
 	
bolge_aci=-3.14
 	
cezeri.don(bolge_aci)
 	
cezeri.bekle(bolge_donus)
 	
elif cezeri.gnss.enlem < varis_bolgesi.enlem and cezeri.gnss.boylam > varis_bolgesi.boylam:
	Soru 2 de de detaylı olarak anlattığımız bu fonksiyon sayesinde aracımızın olduğu konumu bir koordinat düzleminin merkezine koyduktan sonra hedefin bulunduğu bölgeye göre yaptığımız işlemler sonucunda aracımızın dönmesi gereken yönünü belirliyoruz. Bu koşulları hedefin enlem ve boylam bilgilerinin aracın başlangıç konumunun enlem ve boylam bilgileri ile karşılaştırılması sonucunda ulaşıyoruz. Hedef ile araç arasındaki yatay taraftaki dar açıyı yukarıdaki işlemler sayesinde öğreniyoruz. Eğer bölgemiz birinci bölgede ise saatin tersi yönünde hesapladığımız derece kadar dönmesini sağlıyoruz. İkinci bölgede ise yine aynı mantıkla saatin tersi yönünde,3. ve 4. bölge içinde saat yönünde döndürerek aracın hedefe en kısa şekilde dönmesini sağlıyoruz. if fonksiyonunda kullandığımız “bolge_donus” değişkenimiz aracın dönmesi gereken süreyi tutarken “bolge_aci” değişkenimiz dönmesi gereken yönü (saat yönü veya saatin tersi yönü) olmak üzere tutmaktadır. Hedefe ulaştıktan sonra bu değerleri “yon_duzelt()” fonksiyonuna gönderiyoruz. Bu sayede sıradaki hedeflere sorunsuz bir şekilde gidebiliyoruz.

bolge_donus=(4.0*(360-(360.0-derece_aci)))/345.0 bolge_aci=-3.14 cezeri.don(bolge_aci) cezeri.bekle(bolge_donus)	

Soru 10.3 Hava aracının gittiği yönü hesaplayan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

ALGORİTMA	AÇIKLAMA

while ((abs(cezeri.gnss.enlem-varis_bolgesi.enlem)>=0.64) or (abs(cezeri.gnss.boylam-varis_bolgesi.boylam)>=0.64)):
cezeri.ileri_git(cezeri.hizli)
bas_acisi=(345.0*bolge_donus)/4
if bolge_aci<0:
bas_acisi=(-1*bas_acisi)
print(bas_acisi)
cezeri.bekle(0.5)
	Hava aracımızın her hedefe gittiğinde gitmesi gereken yönü derece olarak yazdıran kodumuz bu şekildedir. While yapısı içerisinde sürekli olarak hesapladığımız bu değeri "bas_acisi" değişkeninde tutuyoruz. Dönmemiz gereken yön için beklememiz gereken süre değişkenimiz bir önceki soruda bahsettiğimiz "bolge_donus" değişkenimizdi. Bu değişkenimizi tersine bir oran ile açı değerine dönüştürüyoruz. Süre değişkenimiz her zaman pozitif değerde olduğu için "bolge_aci" değişkenimizden de faydalanarak aracımızın yönelim durumunun eksi yöne mi artı yöne mi olduğunu kararlaştırıyoruz ve 0,5 saniye aralıklarla aracımızın gittiği yönü ekranımıza yazdırıyoruz.

Soru 11 Senaryo 16'da şarj durumunu gözeterek şarj istasyonlarını tespit ettiğiniz ve hava aracını oraya yönlendirdiğiniz algoritma için aşağıdaki soruları cevaplayınız.

Soru 11.1 Batarya durumlarını hangi veri yapısından öğrendiniz?

Aracın bataryasının doluluk bilgisini "cezeri.batarya.veri" yapısını kullanarak öğrenmekteyiz.

Soru 11.2 Batarya durumunun hangi gerilim eşiğinde şarj istasyonuna inilmesi gerektiğini seçtiniz? Neden?

Aracı şarj etmek için 3 farklı gerilim aralığı seçerek aracı indirmeye karar verdik. Bu gerilim noktalarımız bize en yakın olan şarj istasyonuna olan uzaklığımıza ve şarj aralığına göre değişmektedir. En yakın olan şarj istasyonuna olan uzaklığımız ile aracın konumu arasındaki mesafeyi her zaman kontrol ediyoruz. Yukarıda bahsettiğimiz 3 farklı durumdan ilki elde ettiğimiz uzaklık değeri 17'den büyükken ve şarj durumumuz 48'e eşit ve küçük ise, ikincisi uzaklık değerimiz 15 ten küçük ve şarjımız 40'a eşit ve küçük ise, üçüncü koşulumuz ise uzaklık değerimiz 5'e eşit ve küçük , şarjımız 65'e eşit ve küçükse en yakın şarj istasyonuna yönlendirmekteyiz.

Bu koşullardan 1. ve 2. Koşulda seçtiğimiz gerilim değerlerini seçme nedenimiz cezeri hızlı giderken 1 metrede yaklaşık 1.25 şarj harcamasıdır. Bu değere senaryolar üzerinde deneyerek ulaştık. Ve yapılan hesaplamalar sonucunda bu değer aralıklarını şarj noktasına giderken engel ile karşılaşmasını da hesaba katarak ilk iki koşul için uygun gördük. Üçüncü koşulda da şarj istasyonuna çok yakınından geçtiği durumlarda şarjı belirlediğimiz değerden küçük ise kendini şarj istasyonuna götürmesini istedik. Bu sayede daha az zaman harcayarak hedeflere ulaşmasını sağladık.

Soru 11.3 Şarj istasyonlarının olduğu veri yapısı nedir?

"cezeri.harita.sarj_istasyonlari" ifadesi harita üzerinde bulunan bütün şarj istasyonların saklandığı dizidir. Dizinin içinde bulunan şarj istasyonları verilerine for döngüsü kullanarak " for sarj_istasyonu in cezeri.harita.sarj_istasyonlari : " yapısını kullanarak her bir şarj istasyonun konum verilerine for döngüsünün içinde "sarj_istasyonu.enlem","sarj_istasyonu.boylam", komutları ile ulaşmaktayız. Şarj istasyonlarının komut bilgilerine bu şekilde ulaşıyoruz.

Kaynakça

aligören.com . (2017, 1 29). aligören.com Web sitesi : https://www.google.com/amp/s/aligoren.com/pythonda-struct-kullanimi/amp/ adresinden alındı

Linkedin. (2022, 7 3). Linkedin Web sitesi: https://www.linkedin.com/pulse/uydularla-konum-belirleme-sistemlerinin-siber-g%C3%BCvenli%C4%9Fi-nazli-can/?originalSubdomain=tr adresinden alındı

openaccess.hacettepe.edu.tr. (2022, 7 3). openaccess.hacettepe.edu.tr Web sitesi : http://openaccess.hacettepe.edu.tr:8080/xmlui/bitstream/handle/11655/2717/4994fa44-46e4-4fd8-8fd4-31403ff62d09.pdf?sequence=1&isAllowed=y adresinden alındı

Riders. (2020, 1 1). Riders Web sitesi: https://riders.ai/arena/riders-robotik-ligi?source=google&medium=ppc&term=funnel_3&campaignid=17471238899&adgroupid=143388772692&keyword=riders&device=c&gclid=CjwKCAjw_ISWBhBkEiwAdqxb9qyQ7QdFXNrJgvbDCJYPwi-7kRhWnhRkIILXW_v7BeI1ngS-spXWHhoC-DMQAvD_BwE# adresinden alındı

stringfixer. (20222, 7 3). stringfixer Web sitesi: https://stringfixer.com/tr/Pressure_transmitter adresinden alındı